

# EDA

March 18, 2026

## 0.0.1 Exploratory Data Analysis (EDA)

**Exploratory Data Analysis (EDA)** is the process of analyzing and understanding a dataset before applying any machine learning or statistical model.

It helps us:

- Understand data structure
- Detect patterns and relationships
- Identify missing values and outliers
- Validate assumptions
- Prepare data for further analysis

EDA is one of the **most important steps** in any data science workflow.

---

## 2. EDA Workflow (Step-by-Step)

### Step 1: Understand the Data

- Load the dataset
- Check shape (rows, columns)
- View first/last rows
- Understand column names and types

### Step 2: Data Cleaning

- Handle missing values
- Fix incorrect data types
- Remove duplicates
- Standardize formats

### Step 3: Univariate Analysis

Analyze **one variable at a time**

- Numerical: mean, median, min, max
- Categorical: value counts

### Step 4: Bivariate Analysis

Analyze **relationship between two variables**

- Numerical vs Numerical → correlation
- Categorical vs Numerical → groupby analysis
- Categorical vs Categorical → cross-tab

### Step 5: Multivariate Analysis

Analyze **multiple variables together**

- Correlation matrix
- Pair plots
- Feature interactions

### Step 6: Visualization

- Histograms
- Boxplots
- Scatter plots
- Bar charts
- Heatmaps

---

### *Basic Setup*

“python import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns

```
[1]: #RUN THIS CELL
import requests
from IPython.core.display import HTML
styles = requests.get("https://raw.githubusercontent.com/Harvard-IACS/
↳2018-CS109A/master/content/styles/cs109.css").text
HTML(styles)
```

```
[1]: <IPython.core.display.HTML object>
```

```
[ ]:
```

```
[2]: # import the necessary libraries
%matplotlib inline
import numpy as np
import scipy as sp
import matplotlib as mpl
import matplotlib.cm as cm
import matplotlib.pyplot as plt
import pandas as pd
import time
pd.set_option('display.width', 500)
pd.set_option('display.max_columns', 200)
pd.set_option('display.notebook_repr_html', True)
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

import warnings
warnings.filterwarnings('ignore')
%config InlineBackend.figure_format = 'retina'
```

```
[3]: titanic= pd.read_csv("titanic.csv")
titanic.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null   int64
1   pclass          891 non-null   int64
2   sex             891 non-null   str
3   age            714 non-null   float64
4   sibsp          891 non-null   int64
5   parch          891 non-null   int64
6   fare           891 non-null   float64
7   embarked       889 non-null   str
8   class          891 non-null   str
9   who            891 non-null   str
10  adult_male     891 non-null   bool
11  deck          203 non-null   str
12  embark_town    889 non-null   str
13  alive         891 non-null   str
```

```
14 alone          891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 92.4 KB
```

```
[4]: titanic.columns
```

```
[4]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked',
'class', 'who', 'adult_male', 'deck', 'embark_town', 'alive', 'alone'],
dtype='str')
```

```
[5]: # drop columns
```

```
columns = ['embarked', 'who', 'adult_male', 'embark_town', 'alive', 'alone']

titanic = titanic.drop(columns=columns)
titanic
```

```
[5]:
```

|     | survived | pclass | sex    | age  | sibsp | parch | fare    | class  | deck |
|-----|----------|--------|--------|------|-------|-------|---------|--------|------|
| 0   | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | Third  | NaN  |
| 1   | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | First  | C    |
| 2   | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | Third  | NaN  |
| 3   | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | First  | C    |
| 4   | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | Third  | NaN  |
| ..  | ...      | ...    | ...    | ...  | ...   | ...   | ...     | ...    | ...  |
| 886 | 0        | 2      | male   | 27.0 | 0     | 0     | 13.0000 | Second | NaN  |
| 887 | 1        | 1      | female | 19.0 | 0     | 0     | 30.0000 | First  | B    |
| 888 | 0        | 3      | female | NaN  | 1     | 2     | 23.4500 | Third  | NaN  |
| 889 | 1        | 1      | male   | 26.0 | 0     | 0     | 30.0000 | First  | C    |
| 890 | 0        | 3      | male   | 32.0 | 0     | 0     | 7.7500  | Third  | NaN  |

```
[891 rows x 9 columns]
```

```
[6]: # Find for how many passengeres we do not have
# their deck information.
missing_decks = len(titanic[(pd.isna(titanic['deck']) == True)])
missing_decks
```

```
[6]: 688
```

```
[7]: # Plotting one variable's distribution(categorical and continuous)
# What was the age distribution among passengers in the Titanic?
# Set seaborn style for better visuals (adds grid background)
sns.set(style="whitegrid")

# Create a figure with specific size (width=8, height=3)
plt.figure(figsize=(8, 3))

# Plot histogram of passenger ages
```

```

# bins=20 → divides age into 20 intervals
# color → sets bar color
sns.histplot(data=titanic, x="age", bins=20, color="skyblue")

# Set x-axis range (age from 0 to 90)
plt.xlim(0,90)

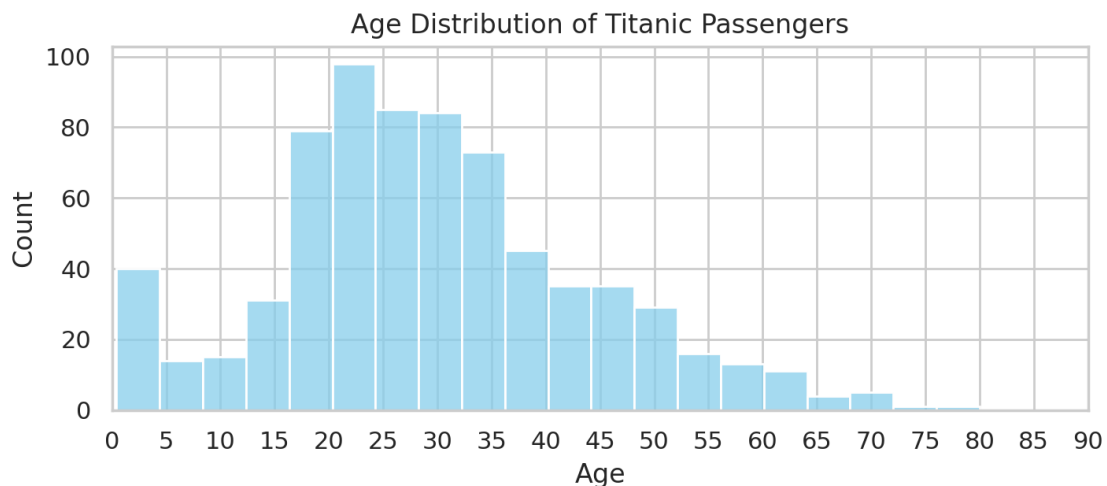
#Set x-axis ticks at interval of 5
plt.xticks(range(0, 91, 5))

# Add labels to axes
plt.xlabel("Age")
plt.ylabel("Count")

# Add title to the plot
plt.title("Age Distribution of Titanic Passengers")

# Display the plot
plt.show()

```



```

[8]: # Create a figure (f) and axes (ax)
# figsize=(8,3) → width=8 inches, height=3 inches
f, ax = plt.subplots(1, 1, figsize=(8, 3))

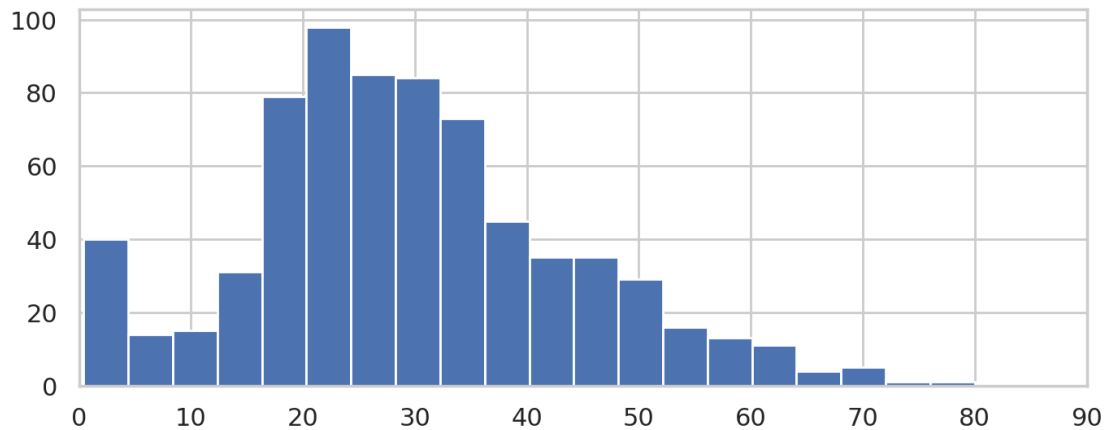
# Plot histogram on the axes
# titanic.age → age column
# bins=20 → divide age data into 20 intervals (bars)
ax.hist(titanic.age, bins=20)

# Set x-axis limits from 0 to 90

```

```
# This controls the visible range of ages
ax.set_xlim(0, 90)
```

[8]: (0.0, 90.0)



[9]: # Count all the infants on board (age < 3) and all the children ages 5-10.

```
infants = len(titanic[(titanic.age < 3)])
children = len(titanic[(titanic.age >= 3) & (titanic.age < 10)])
print(f'There were {infants} infants and {children} children on board the_
↳Titanic')
```

There were 24 infants and 38 children on board the Titanic

```
[10]: from matplotlib.patches import Patch #

# Get a colormap (a set of colors)
cmap = plt.get_cmap('Pastell1')

# Select specific colors from the colormap (values between 0 and 1)
young = cmap(0.5)    # color for first group
middle = cmap(0.2)   # color for second group
older = cmap(0.8)    # color for third group

# Create figure and axis
fig, ax = plt.subplots()

# Plot histogram
# y_values → counts in each bin
# bins → bin edges
# patches → list of bar objects (each bar is one bin)
y_values, bins, patches = ax.hist(titanic.age, bins=10)
```

```

# Change colors of specific bins

# First bin
for i in range(0, 1):
    patches[i].set_facecolor(young)

# Second and third bins
for i in range(1, 3):
    patches[i].set_facecolor(middle)

# Remaining bins
for i in range(3, 10):
    patches[i].set_facecolor(older)

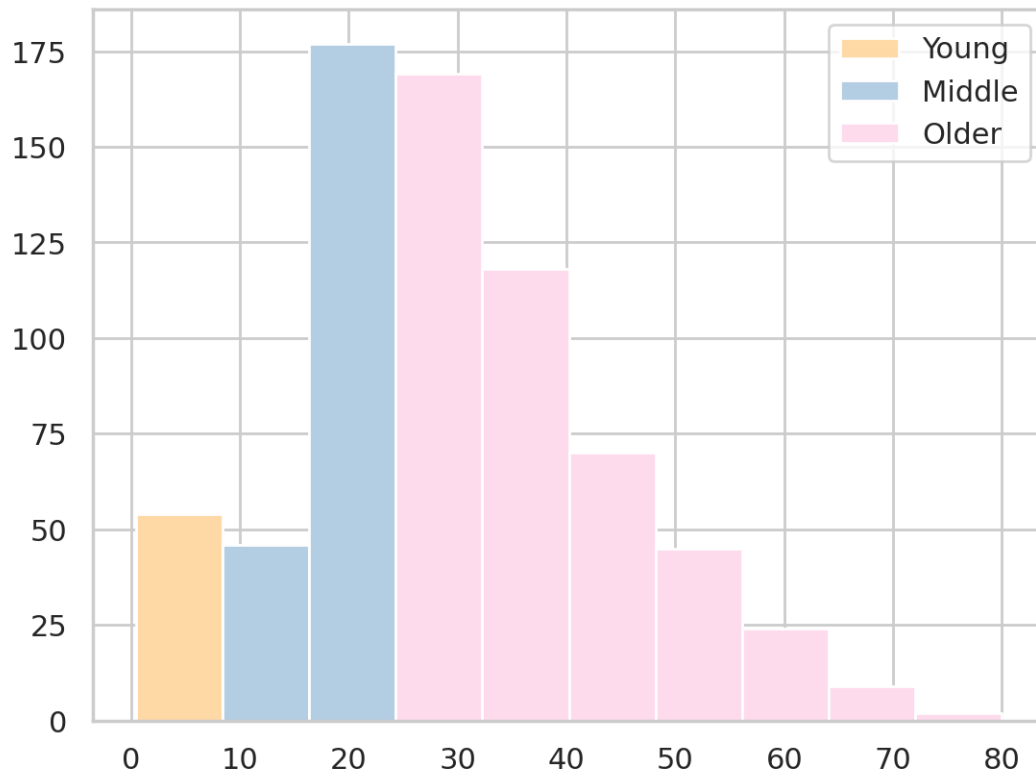
legend_elements = [
    Patch(facecolor=young, label='Young'),
    Patch(facecolor=middle, label='Middle'),
    Patch(facecolor=older, label='Older')
]

# Add legend to plot
ax.legend(handles=legend_elements)

# Add grid
ax.grid(True)

# Show plot
plt.show()

```



```
[11]: # or

# # Define real age groups (bins)
# bins = [0, 18, 40, 80]

# # Labels for legend
# labels = ["Children (0-18)", "Adults (18-40)", "Older (40+)"]

# # Colors for each group
# colors = ["skyblue", "orange", "red"]

# # Create plot
# fig, ax = plt.subplots()

# # Plot histogram with multiple groups
# for i in range(len(bins) - 1):
#     # Filter data for each age group
#     data = titanic[(titanic.age >= bins[i]) & (titanic.age < bins[i+1])].age

#     # Plot each group separately
#     ax.hist(data, bins=10, color=colors[i], label=labels[i], alpha=0.7)
```

```
# # Add legend (color box with labels)
# ax.legend()

# # Labels and title
# ax.set_xlabel("Age")
# ax.set_ylabel("Count")
# ax.set_title("Age Distribution by Group")

# plt.show()
```

**Kernel Density Estimation (KDE)** is a statistical method used to estimate the **probability density function (distribution)** of a dataset by creating a **smooth continuous curve** based on the data points.

---

### Important Points

- KDE provides a **smooth representation** of data distribution
- It is an alternative to a **histogram**
- Uses **kernels (small curves)** placed on each data point
- **Bandwidth (bw)** controls the smoothness of the curve
- Small bandwidth → more detail (noisy curve)
- Large bandwidth → smoother curve (may hide details)
- Default bandwidth is usually chosen automatically
- Helps identify:
  - Shape of distribution
  - Peaks (high density areas)
  - Skewness
- Works best with **continuous numerical data**

---

### Types of Distribution Shapes    1. *Normal Distribution*

- Symmetrical (bell-shaped curve)
- Mean    Median    Mode

- No skewness
- Most values are concentrated around the center

---

### 2. *Left Skewed Distribution (Negative Skew)*

- Tail extends toward the **left side**
- Most values are on the **right side**
- Mean < Median

---

### 3. *Right Skewed Distribution (Positive Skew)*

- Tail extends toward the **right side**
- Most values are on the **left side**
- Mean > Median

```
[12]: # Import seaborn (for plotting)
import seaborn as sns

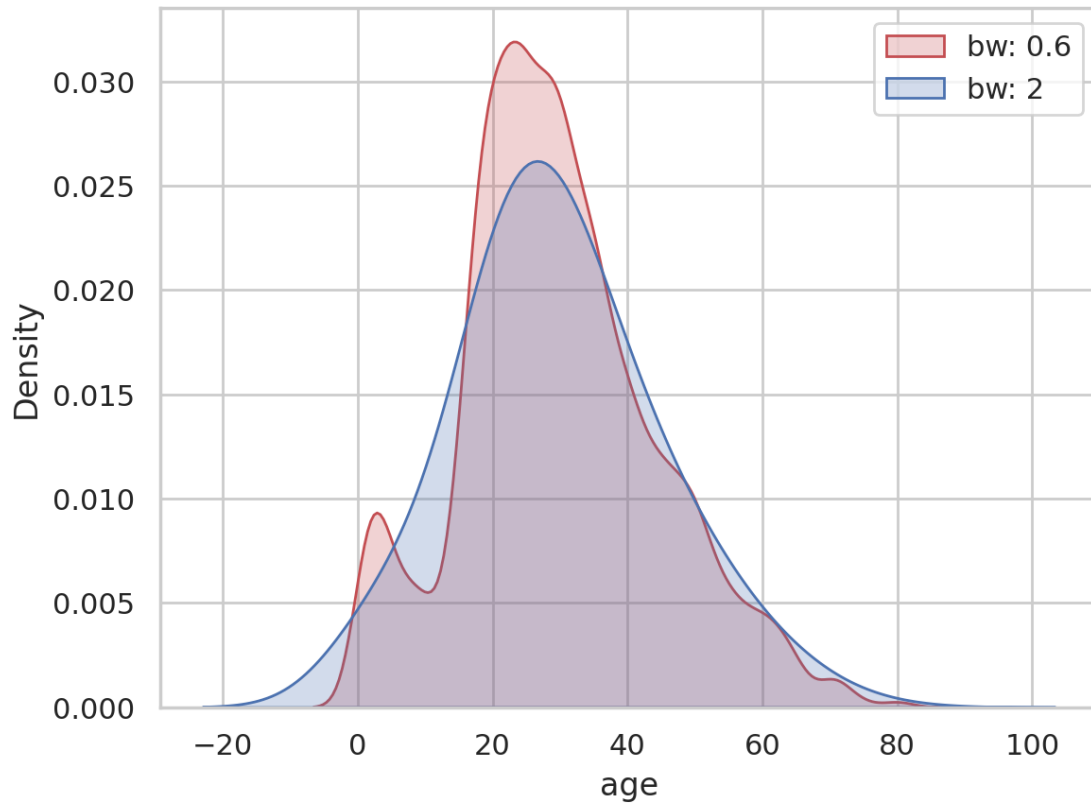
# KDE Plot 1 (More detailed curve)
# bw=0.6 → smaller bandwidth → more detailed (wiggly curve)
# label → used in legend
# shade=True → fills the area under the curve
# color="r" → red color
sns.kdeplot(titanic.age, bw_adjust=0.6, label="bw: 0.6",
            color="r", fill=True)

# KDE Plot 2 (Smoother curve)
# bw=2 → larger bandwidth → smoother curve (less detail)
# shade=True → fills the area
# no color specified → default color will be used
sns.kdeplot(titanic.age, bw_adjust=2, label="bw: 2"

            ,fill=True)

# Add legend to differentiate both curves
plt.legend()

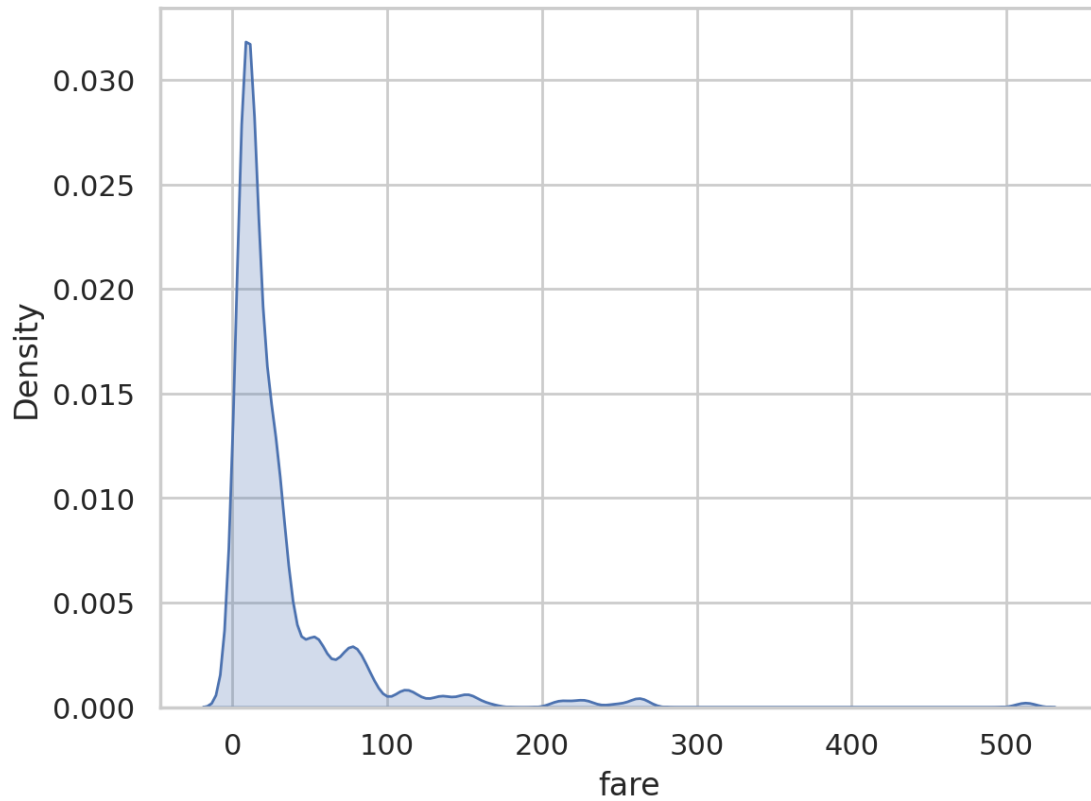
# Show the plot
plt.show()
```



```
[13]: # finding: not perfectly skewed, high bw= smooth curve
```

```
[14]: # Plot the distribution of fare paid by passengers
# bw_adjust=0.5 → smaller bandwidth → more detailed (captures small variations)
# label → used for legend
# shade=True → fills the area under the curve

sns.kdeplot(titanic.fare, bw_adjust=0.5, label="bw: 0.5", shade=True);
```



```
[15]: import seaborn as sns
sns.set(color_codes=True)

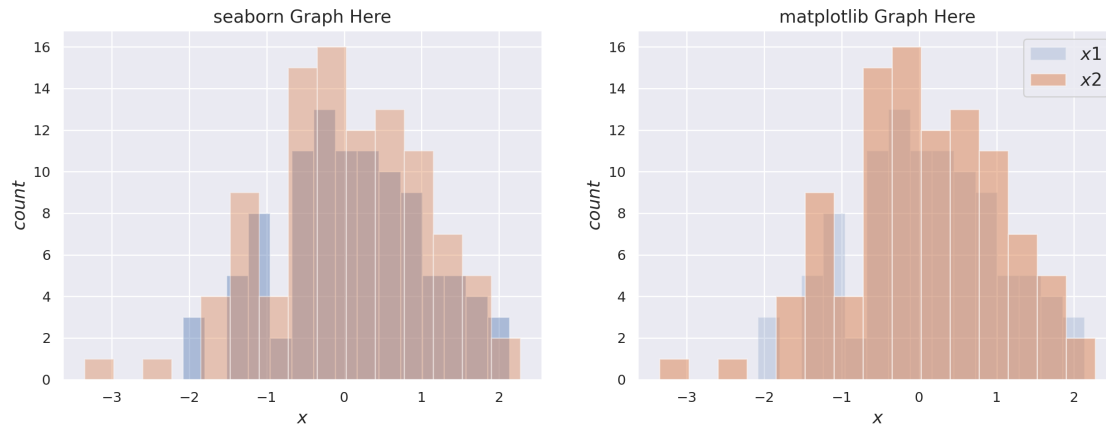
x1 = np.random.normal(size=100)
x2 = np.random.normal(size=100)

fig, ax = plt.subplots(1,2, figsize=(15,5))

# seaborn goes in first subplot
sns.set(font_scale=0.5)
sns.distplot(x1, kde=False, bins=15, ax=ax[0]);
sns.distplot(x2, kde=False, bins=15, ax=ax[0]);
ax[0].set_title('seaborn Graph Here', fontsize=14)
ax[0].set_xlabel(r'$x$', fontsize=14)
ax[0].set_ylabel(r'$count$', fontsize=14)

# matplotlib goes in second subplot
ax[1].hist(x1, alpha=0.2, bins=15, label=r'$x1$');
ax[1].hist(x2, alpha=0.5, bins=15, label=r'$x2$');
ax[1].set_xlabel(r'$x$', fontsize=14)
ax[1].set_ylabel(r'$count$', fontsize=14)
```

```
ax[1].set_title('matplotlib Graph Here', fontsize=14)
ax[1].legend(loc='best', fontsize=14);
```



### ### Box Plot

A **Box Plot (Box-and-Whisker Plot)** is a graphical representation of data distribution using five key values:

- Minimum
- First Quartile (Q1)
- Median (Q2)
- Third Quartile (Q3)
- Maximum

It also highlights **outliers**.

---

#### *Why is it Important?*

- Provides a **quick summary** of data distribution
  - Helps identify **outliers easily**
  - Shows **spread and central tendency**
  - Useful for comparing **multiple groups**
- 

#### *Use Cases*

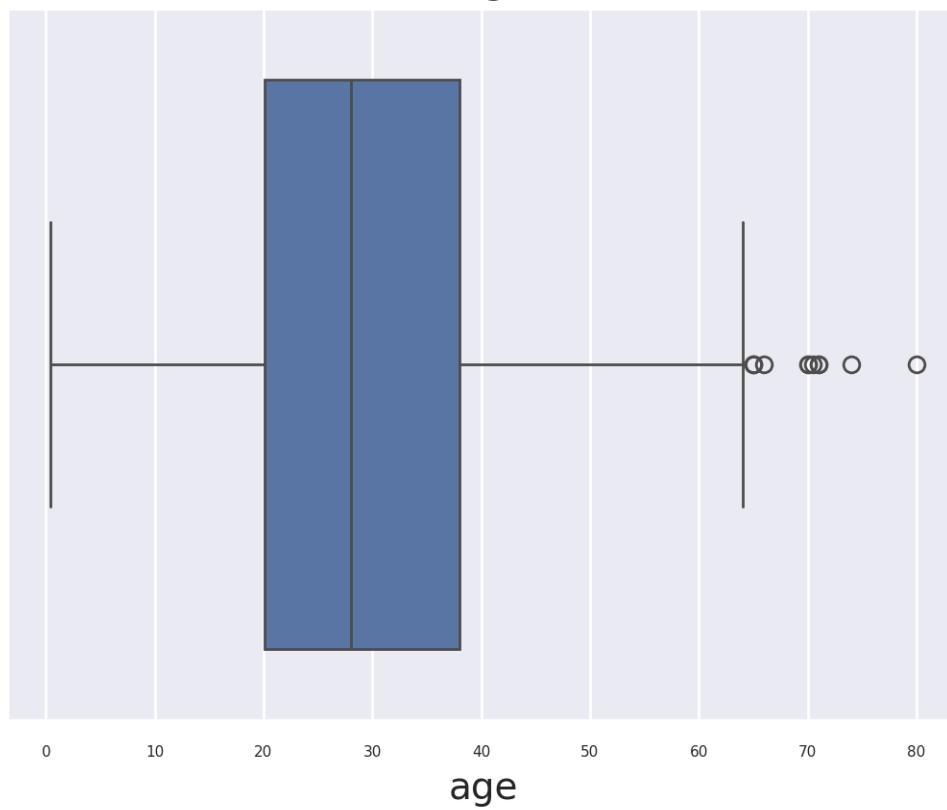
- Comparing distributions across categories (e.g., Salary by Department)
  - Detecting **outliers** in data
  - Understanding **data spread and skewness**
  - Used in **EDA before modeling**
- 

### *Points to Remember*

- The **box** represents the **Interquartile Range (IQR = Q3 - Q1)**
- The **line inside the box** is the **median**
- **Whiskers** extend to normal range of data
- Points outside whiskers are **outliers**
- Skewness can be identified by box position and whisker length
- Works best for **numerical data**

```
[16]: # Box plots
ax = sns.boxplot(x='age', data=titanic)
#ax = sns.boxplot(x=titanic['age']) # another way to write this
ax.set_ylabel(None);
ax.set_xlabel('age', fontsize=14);
ax.set_title('Distribution of age in the Titanic', fontsize=14);
```

Distribution of age in the Titanic



Findings:

Median age is around young-middle age

Most data lies within a moderate range (IQR)

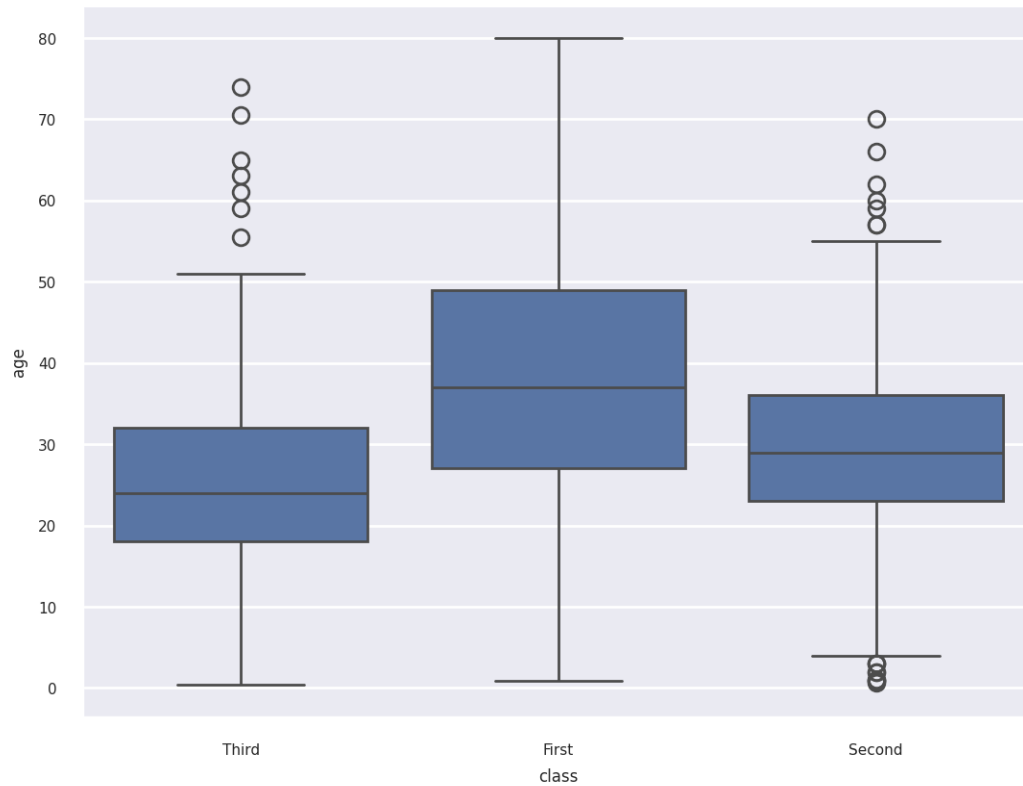
Distribution is right-skewed (longer right whisker)

Presence of outliers on the higher age side

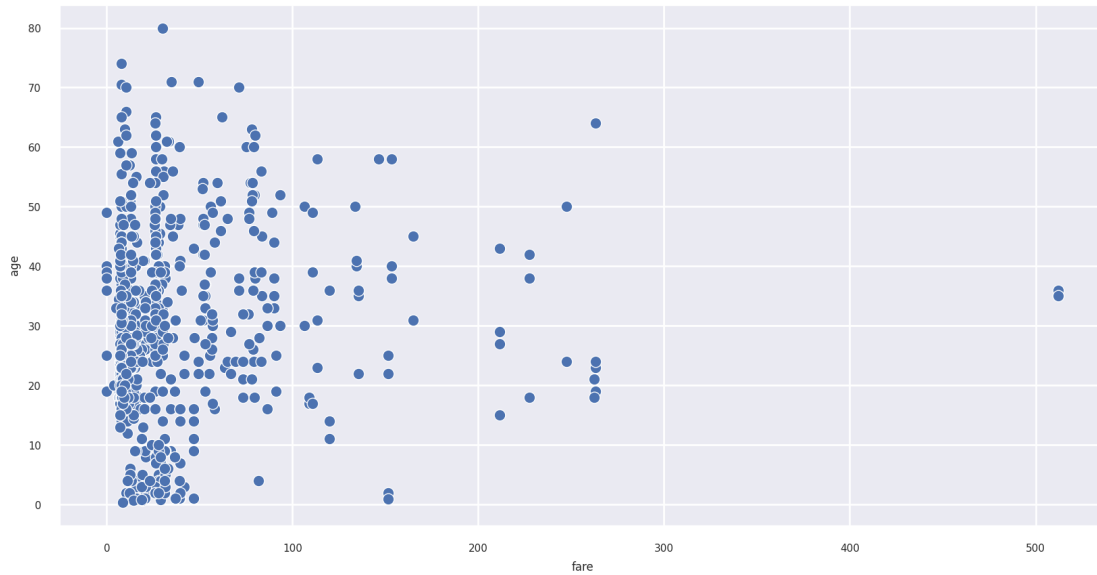
Few extreme values on the lower (younger) side

Majority of passengers are young adults

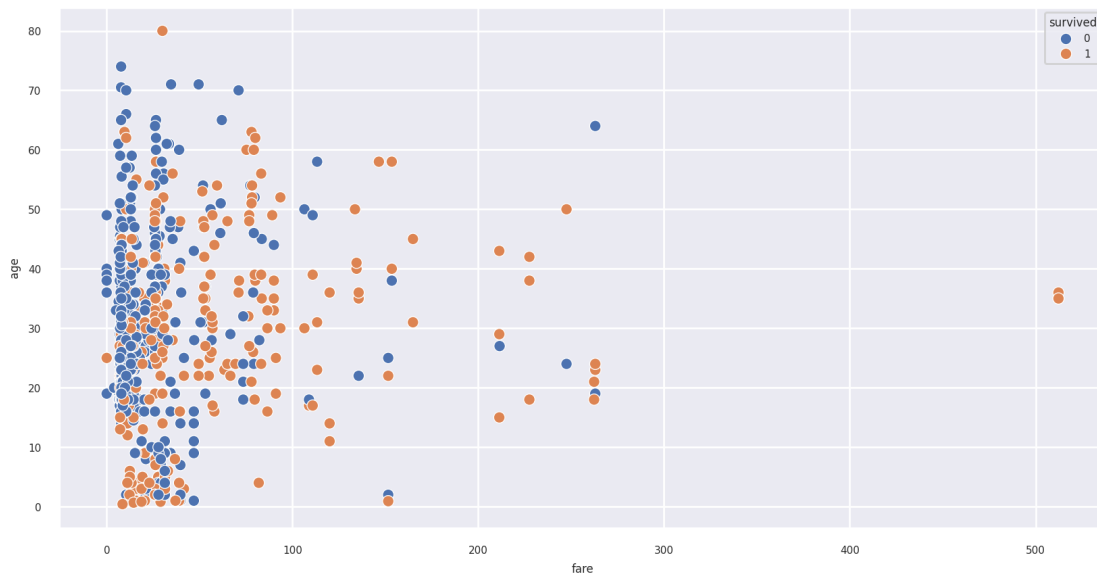
```
[17]: # Did more young people or older ones get  
#      first class tickets on the Titanic?  
  
# two variables seaborn  
ax = sns.boxplot(x='class', y='age', data=titanic)
```



```
[18]: # Scatter plots
      """
      Also called a bivariate distribution
      where each observation is shown with
      a point with x and y values.
      """
      f, ax = plt.subplots(1,1, figsize=(10, 5))
      sns.scatterplot(x="fare", y="age", data=titanic, ax=ax);
```



```
[19]: # Plotting the distribution of three variables
f, ax = plt.subplots(1,1, figsize=(10, 5))
sns.scatterplot(x="fare", y="age", hue="survived", data=titanic, ax=ax);
```



```
[20]: # Plot the distribution of fare paid by passengers according
#       to age, survival and sex.
# Create a figure and axis
```

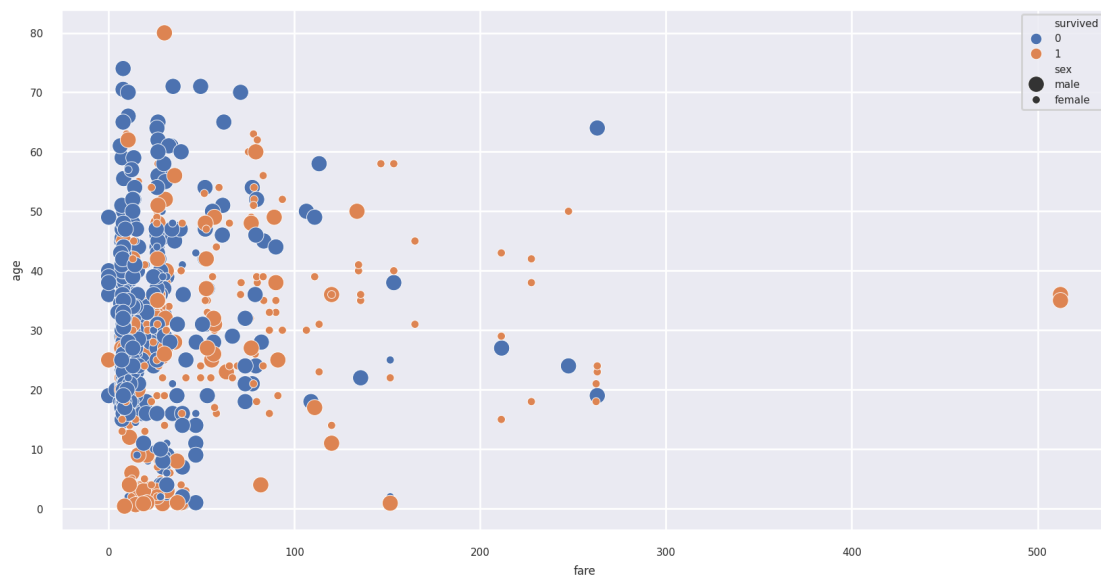
```

# figsize → controls size of plot (width=10, height=5)
f, ax = plt.subplots(1, 1, figsize=(10, 5))

# Scatter plot to analyze relationship between multiple variables
sns.scatterplot(
    x="fare",          # x-axis → ticket fare (price paid)
    y="age",          # y-axis → passenger age
    hue="survived",    # color → survival status (0 = no, 1 = yes)
    size="sex",        # size of points → gender (male/female)
    data=titanic,      # dataset
    ax=ax              # place plot on the created axis
)

# Show the plot
plt.show()

```



findings: Most passengers paid low fares → Dense cluster on the left side (fare < 100)

Higher fare passengers are fewer → Very few points on the right side (fare > 200)

Survival is higher among high-fare passengers → More “survived” points appear in higher fare region

Lower fare passengers show more non-survival → Dense mix, but many non-survivors in low fare range

Age is spread across all fare levels → No strong relationship between age and fare

Gender impact (point size) → Larger points (one gender) appear more among survivors → Suggests gender influenced survival

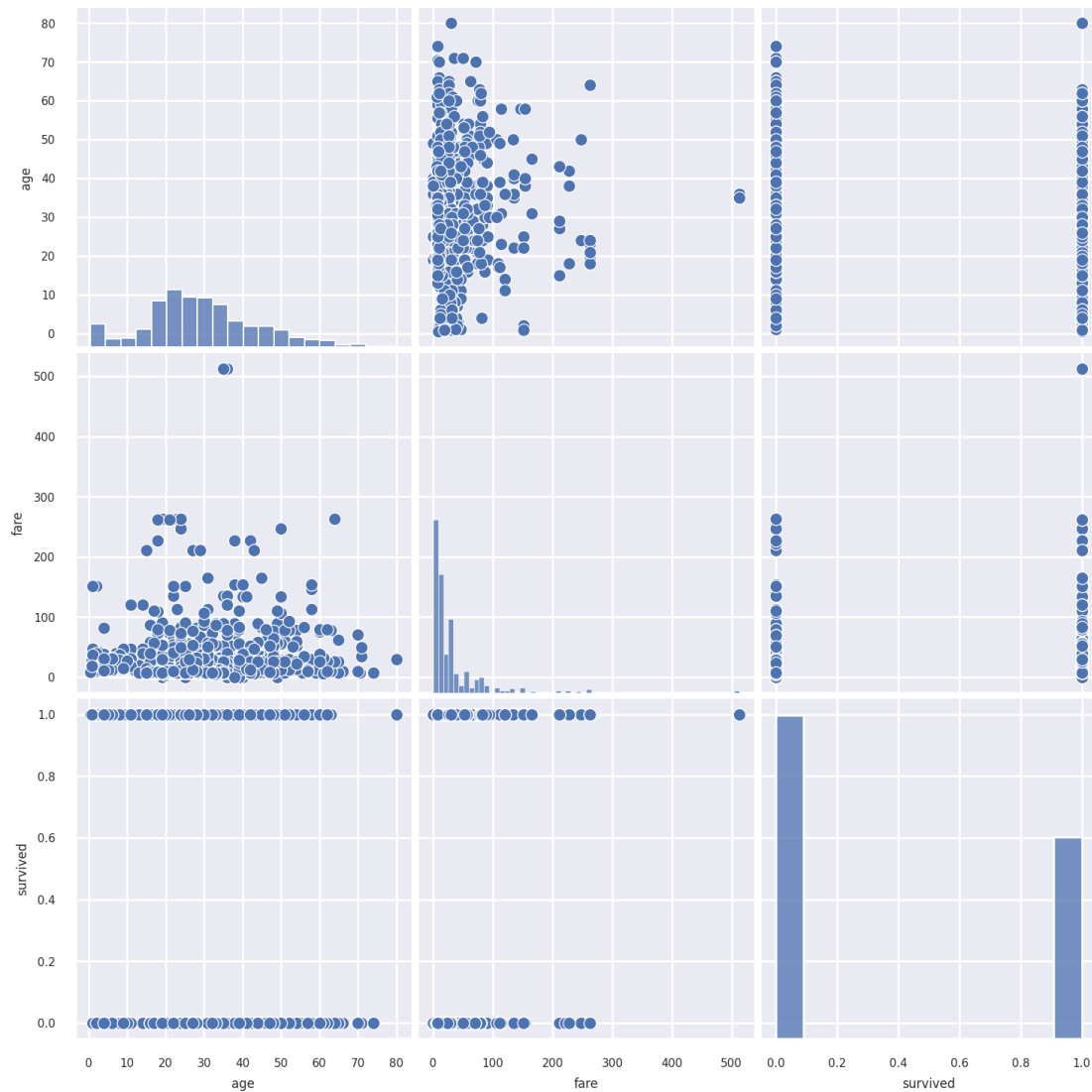
Children present at low fares → Very young ages clustered in low fare region

```
[21]: # Pairplots
      '''
      A pairplot is a powerful visualization tool (commonly
      used in Seaborn) for exploratory data analysis (EDA).
      It helps you understand relationships between
      multiple numerical variables
      in a dataset-all in one grid.
      '''
      titanic.columns

[21]: Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'class',
          'deck'], dtype='str')

[22]: to_plot = ['age', 'fare', 'survived', 'deck']

[23]: df_to_plot = titanic.loc[:,to_plot]
      sns.pairplot(df_to_plot);
```



Findings:

Most passengers are between 20-40 years old

Age distribution is slightly skewed toward younger people

Fare distribution is highly skewed

Most people paid low fares

Few paid very high fares (outliers)

More passengers did not survive than survived

Age vs Fare

No strong relationship

People of all ages paid different fares

Age vs Survival

No strong pattern

Slight trend: children seem to survive more

Fare vs Survival (important)

People who paid higher fares had higher survival chances

Low-fare passengers mostly did not survive

Overall insight

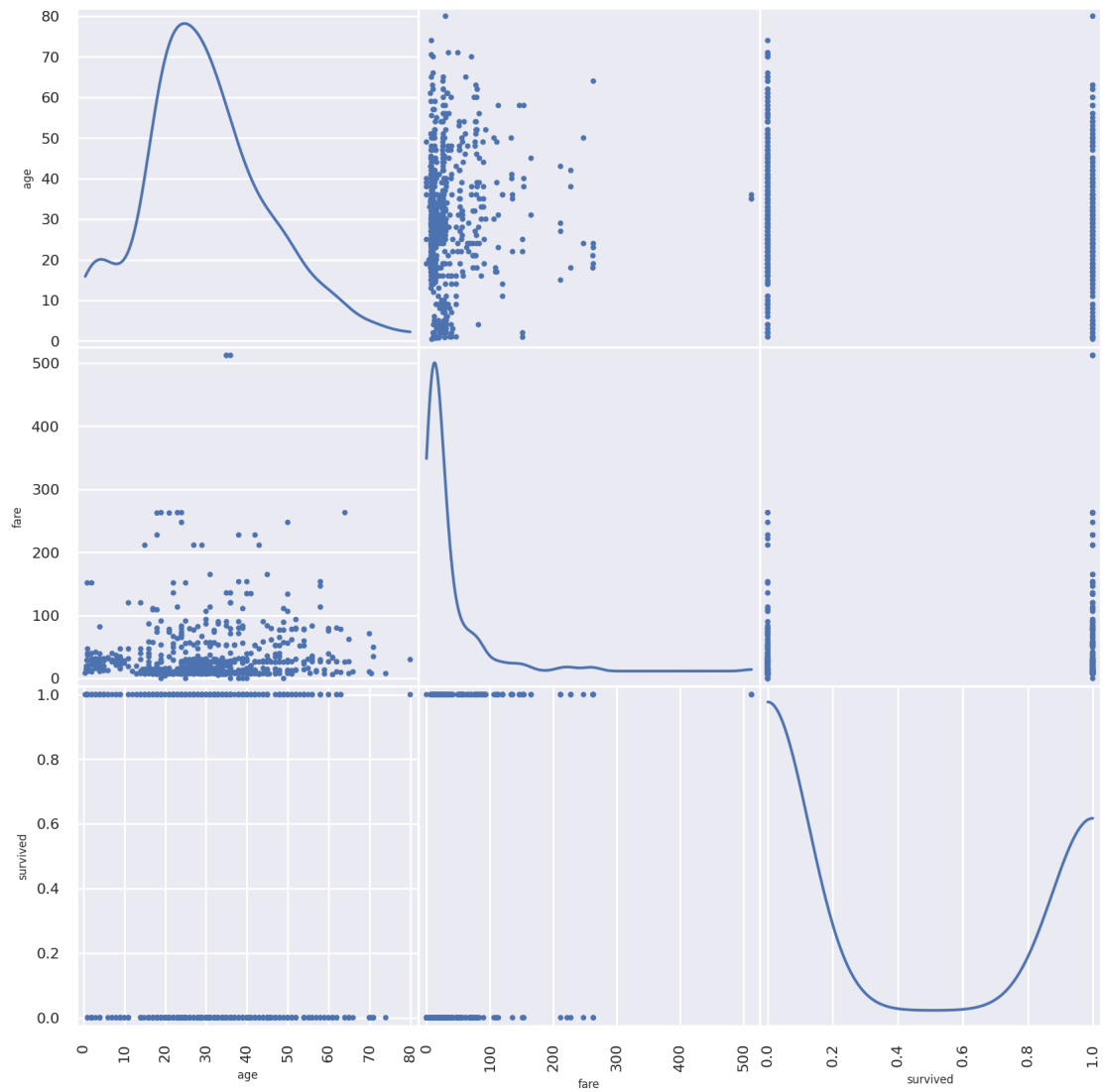
Fare (class/wealth) strongly affects survival

Age has weak impact

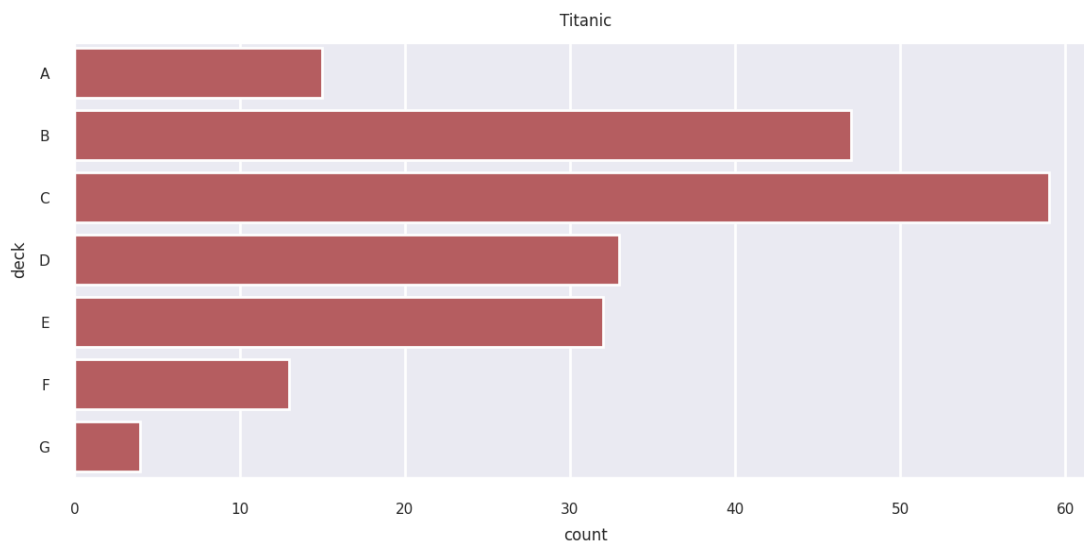
Dataset contains outliers in fare

```
[24]: from pandas.plotting import scatter_matrix # Import scatter_matrix function
      ↪ from pandas

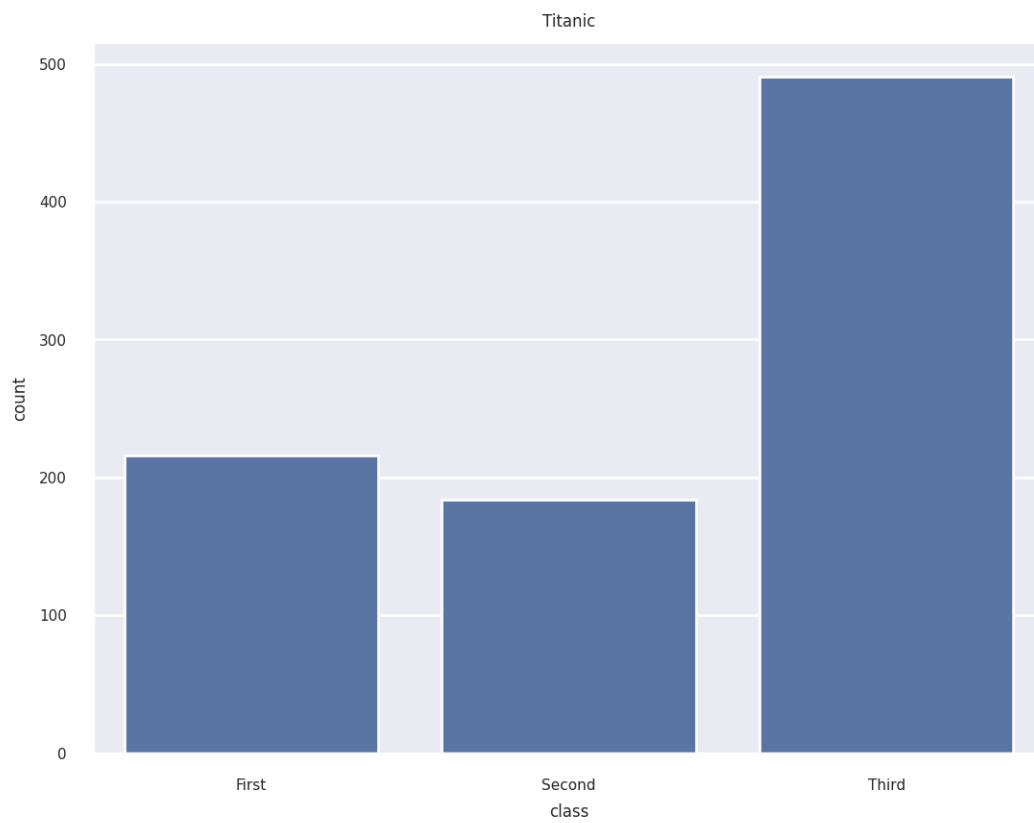
      # Create a scatter matrix (similar to pairplot)
      scatter_matrix(
          df_to_plot,          # DataFrame containing numerical columns to plot
          alpha=1,             # Transparency level (0 = fully transparent, 1 = solid)
          figsize=(10, 10),    # Size of the entire plot (width, height)
          diagonal='kde'       # Plot Kernel Density Estimate on diagonal instead of ↪
          ↪ histogram
      );
```



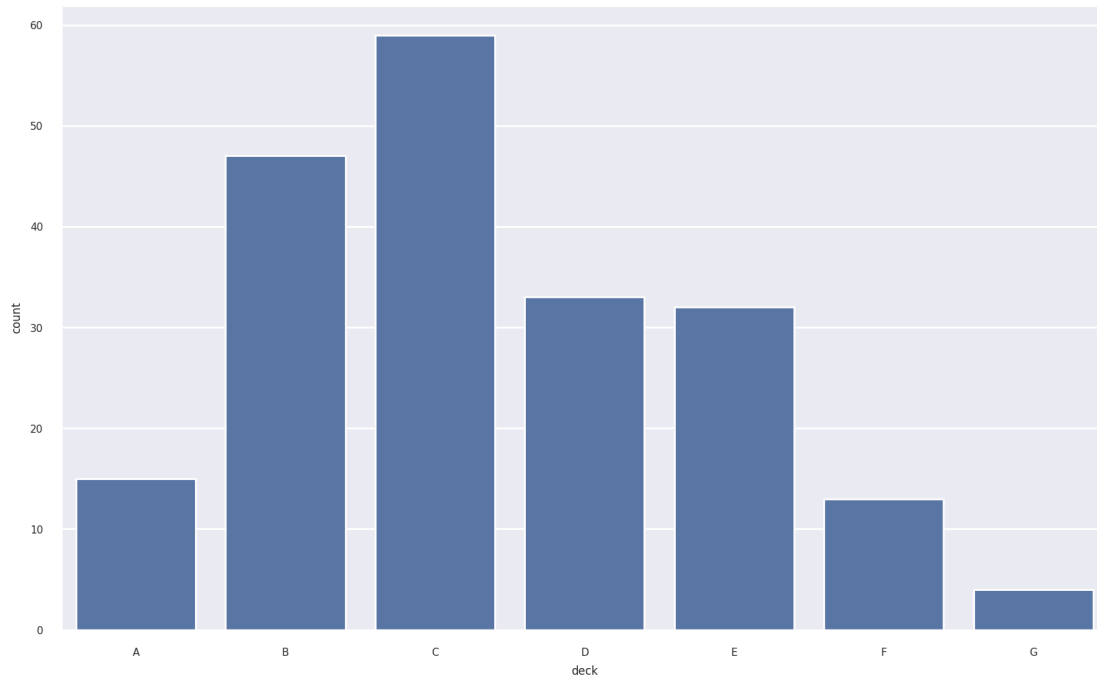
```
[25]: # Plotting Categorical values
titanic = sns.load_dataset('titanic')
f, ax = plt.subplots(figsize=(7, 3));
ax = sns.countplot(y="deck", data=titanic, color="r");
ax.set_title('Titanic');
```



```
[26]: ax = sns.countplot(x="class", data=titanic)  
ax.set_title('Titanic');
```

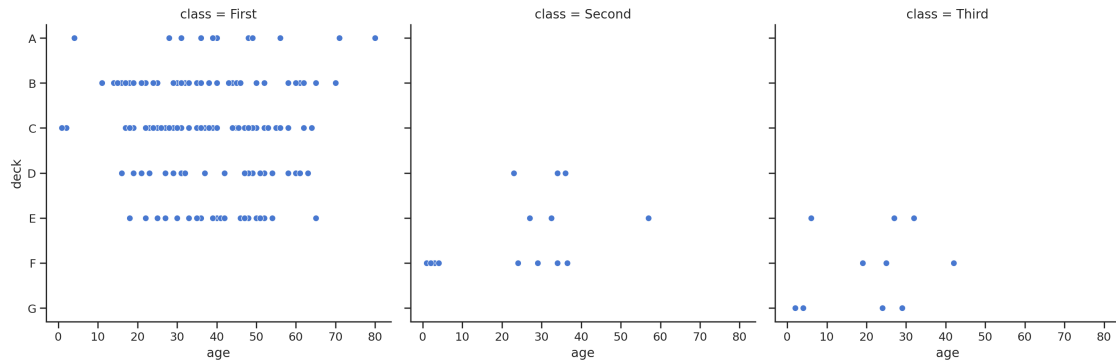


```
[27]: fig, ax = plt.subplots(figsize=(10,6)) # Create figure object
sns.set_context("notebook", font_scale=1, rc={"lines.linewidth": 2.5})
ax = sns.countplot(x="deck", data=titanic)
```



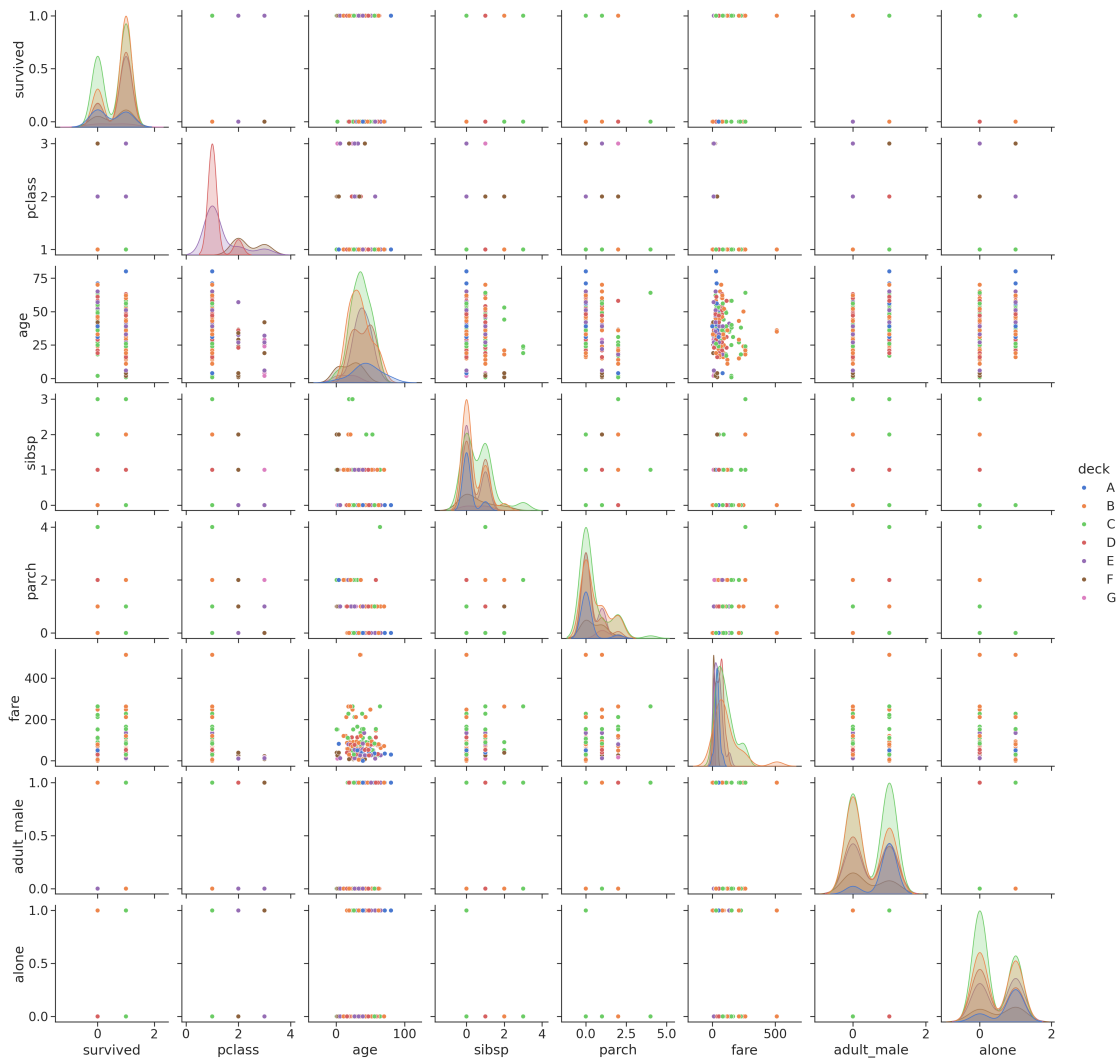
```
[28]: # Set overall theme/style for plots
# relational plot: show relationship between 2 or more variables
sns.set(
    style="ticks",      # Adds ticks (small marks) on axes for a cleaner look
    palette="muted"     # Uses soft/muted colors for better readability
)

# Create a relational plot
sns.relplot(
    x="age",             # X-axis → passenger age
    y="deck",            # Y-axis → deck level (categorical: A, B, C, etc.)
    col="class",         # Create separate plots for each passenger class (1st, 2nd, 3rd)
    data=titanic        # Dataset being used
);
```



```
[29]: # Set the plotting context (controls overall scale of elements)
sns.set_context(
    "notebook",          # Context optimized for notebooks (balanced
    size)                # Increase font size (1.5x larger than default)
    font_scale=1.5,      # Make lines thicker for better visibility
    rc={"lines.linewidth": 2.5}
)

# Create a pairplot
sns.pairplot(
    data=titanic, # Dataset being used
    hue="deck"    # Color-code data points based on deck category (A, B, C,
    etc.)
);
```



## 0.0.2 Pairplot Interpretation (Titanic Dataset with hue=“deck”)

**Survival (*survived*)** - Some decks have higher concentration near 1 (survived) - Others are concentrated near 0 (did not survive) - Indicates deck/location influenced survival chances

**Fare (*fare*)** - Clear pattern: - Higher decks (A, B, C) → higher fares - Lower decks → lower fares  
- Deck is strongly related to ticket class and wealth

**Age (*age*)** - Most passengers fall between 20–40 across all decks - No strong separation by deck - Age is not a key factor for deck classification

**Family Features** (*sibsp*, *parch*) - Most passengers have values close to 0 (traveling alone) - Very few large families - No strong pattern across decks

---

**Adult Male** (*adult\_male*) - Higher concentration of adult males among non-survivors - Some decks show more adult male presence - Gender and age influence survival

---

**Alone** (*alone*) - Majority of passengers are alone (1) - Spread across all decks - No strong relationship with deck

---

**Passenger Class** (*pclass*) - Strong relationship with deck: - Higher decks → 1st class - Lower decks → 2nd and 3rd class - Deck acts as a proxy for class

---

**Key Relationships** - Fare vs Deck → strong relationship - Pclass vs Deck → strong relationship - Survival vs Deck → moderate relationship - Age vs Deck → weak relationship

---

**Final Insights** - Deck is strongly linked to class and fare (wealth) - Higher-deck passengers had better survival chances - Age and family size have weaker influence - Clear socio-economic patterns exist in the dataset

```
[30]: # Line Graph
```

```
[31]: df = pd.DataFrame(np.random.randn(1000, 4), columns=['A', 'B', 'C', 'D'])
df.head()
```

```
[31]:
```

|   | A         | B         | C         | D         |
|---|-----------|-----------|-----------|-----------|
| 0 | 0.938635  | -1.023347 | 2.015129  | 0.125623  |
| 1 | 0.000295  | 0.780000  | 1.415489  | -2.556847 |
| 2 | -0.573476 | 3.458677  | 0.400207  | -0.270858 |
| 3 | 1.814009  | 2.716234  | -0.156311 | -0.866762 |
| 4 | 1.140775  | -0.534883 | 2.414138  | -0.803644 |

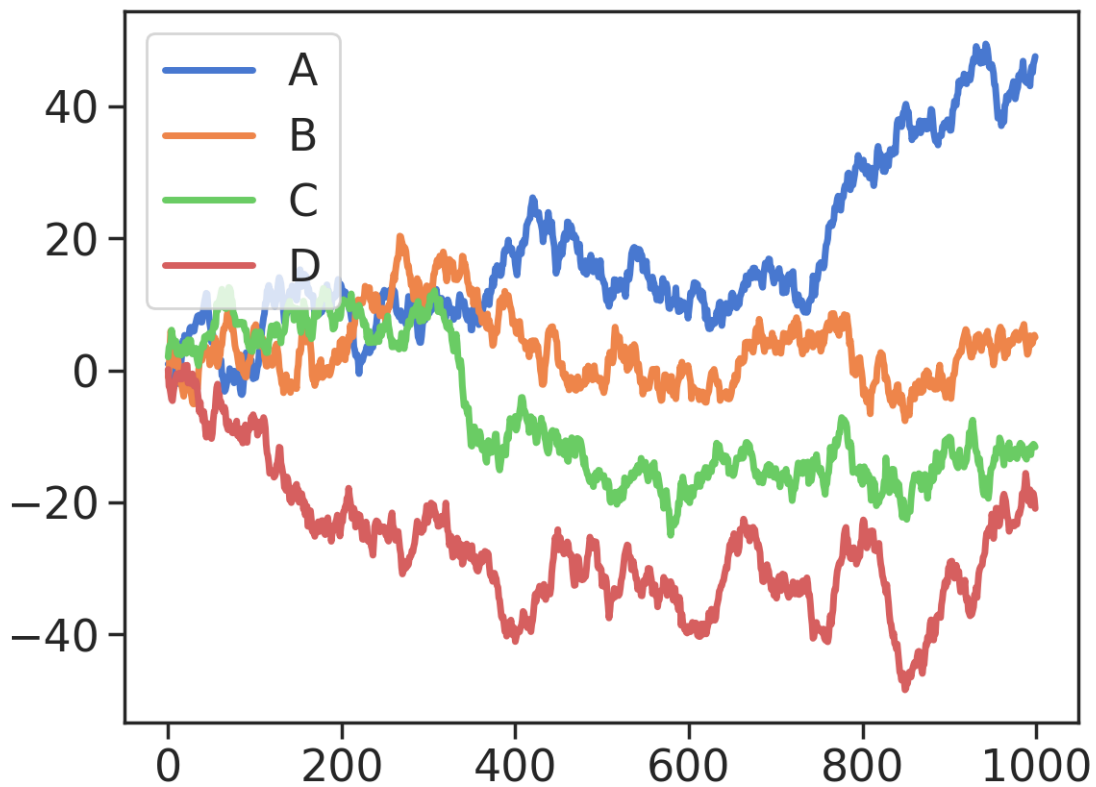
```
[32]: # cumulative sum adds column values as it goes
df = df.cumsum()
df.head()
```

```
[32]:
```

|   | A        | B         | C        | D         |
|---|----------|-----------|----------|-----------|
| 0 | 0.938635 | -1.023347 | 2.015129 | 0.125623  |
| 1 | 0.938929 | -0.243347 | 3.430618 | -2.431224 |
| 2 | 0.365454 | 3.215330  | 3.830825 | -2.702082 |
| 3 | 2.179463 | 5.931564  | 3.674514 | -3.568844 |
| 4 | 3.320237 | 5.396681  | 6.088653 | -4.372488 |

```
[33]: plt.figure();
df.plot();
plt.legend(loc='best');
```

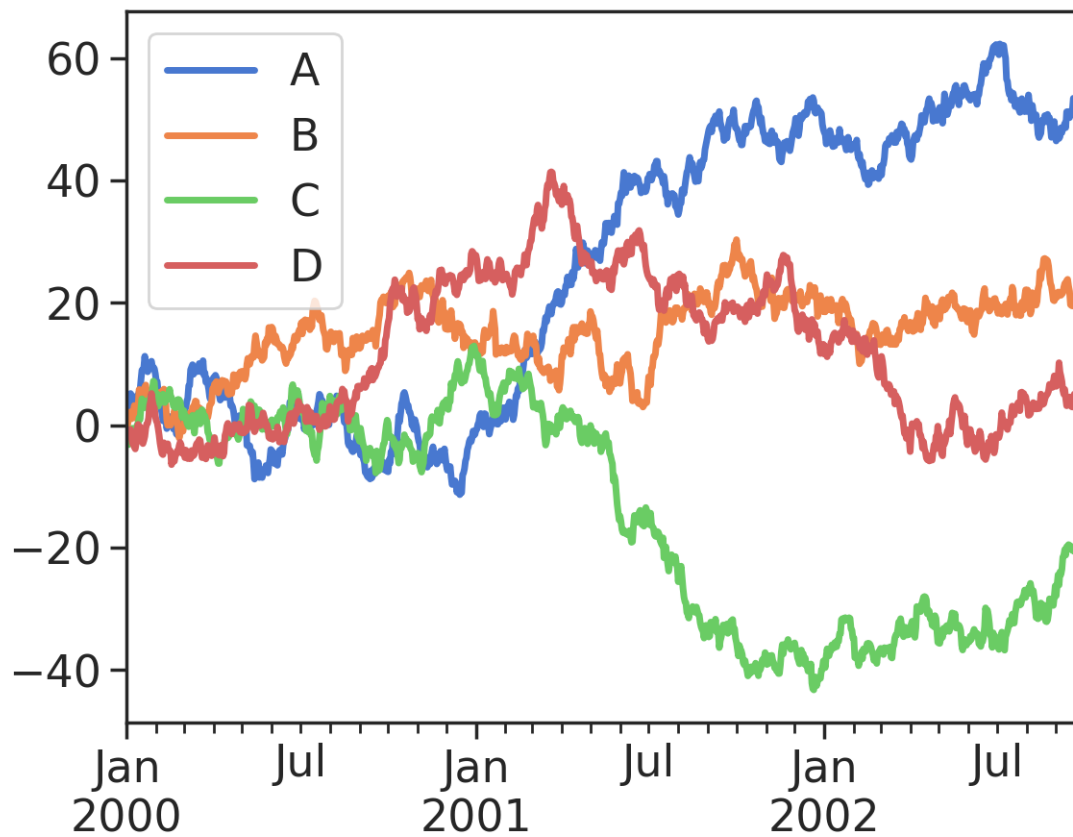
<Figure size 640x480 with 0 Axes>



```
[34]: ts = pd.Series(np.random.randn(1000),
                    index=pd.date_range('1/1/2000', periods=1000))
df = pd.DataFrame(np.random.randn(1000, 4),
                  index=ts.index, columns=list('ABCD'))

df = df.cumsum()
plt.figure();
df.plot();
```

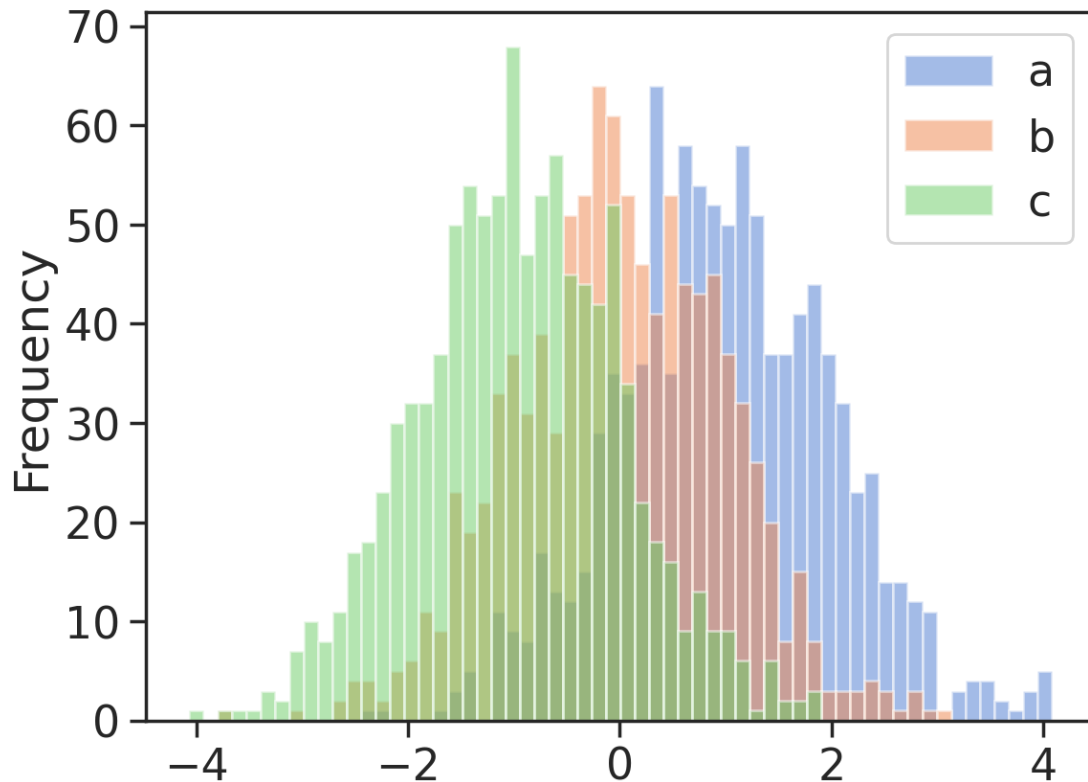
<Figure size 640x480 with 0 Axes>



```
[35]: # Histograms
df4 = pd.DataFrame({'a': np.random.randn(1000) + 1, 'b': np.random.randn(1000),
                    'c': np.random.randn(1000) - 1}, columns=['a', 'b', 'c'])

plt.figure();
df4.plot.hist(alpha=0.5, stacked=False, bins=60);
```

<Figure size 640x480 with 0 Axes>



Three distributions are shown:

Blue → a

Orange → b

Green → c

All three have a bell-shaped curve → Data is normally distributed

Center (mean):

a → around +1 (right side)

b → around 0 (middle)

c → around -1 (left side)

Spread (variance):

All three have similar width → Means similar variability

Overlap:

Distributions overlap in the middle → Some values are common across variables

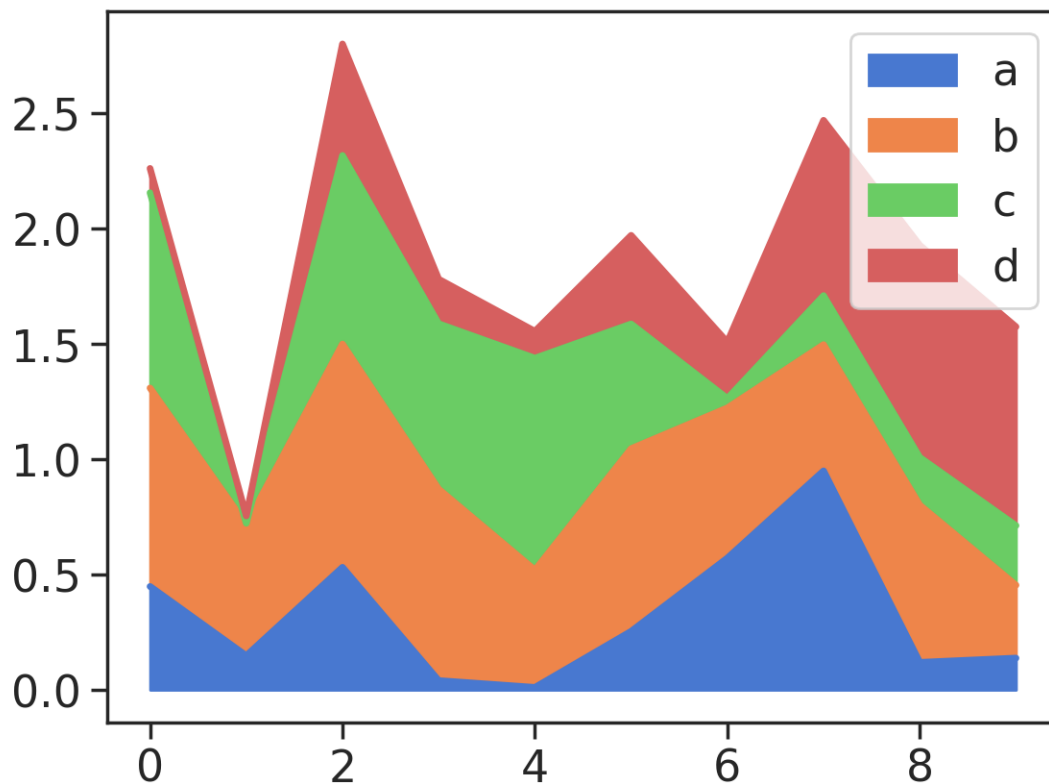
Comparison:

a has highest values

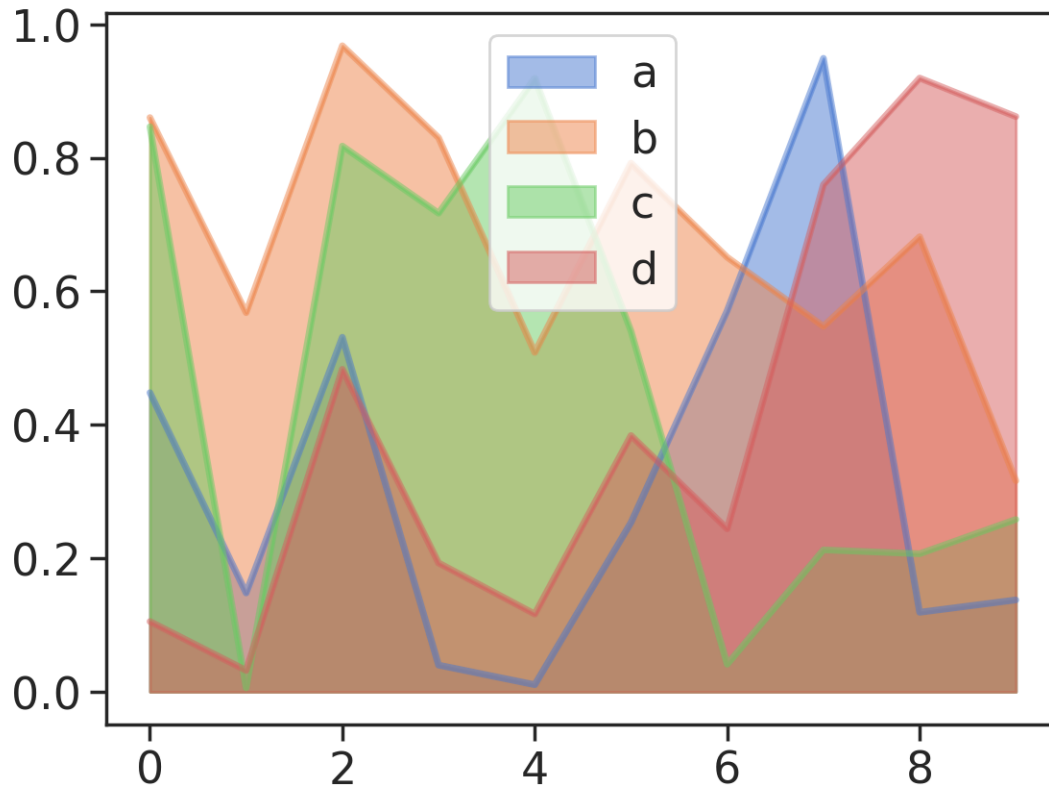
b is moderate

c has lowest values

```
[36]: '''  
A stacked area plot is a graph used to show how multiple variables  
contribute to a total over time (or index).  
  
-Multiple variables are shown  
-Each one is stacked on top of the previous one  
-The top line = total sum  
'''  
df = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b', 'c', 'd'])  
df.plot.area(stacked=True);
```



```
[37]: df.plot.area(stacked=False);
```



### 0.0.3 Final Summary

| Type         | Plot        | Code Example                                                      |
|--------------|-------------|-------------------------------------------------------------------|
| Distribution | histplot    | <code>sns.histplot(data=df, x="age")</code>                       |
|              | kdeplot     | <code>sns.kdeplot(data=df, x="age")</code>                        |
| Relational   | displot     | <code>sns.displot(df["age"])</code>                               |
|              | scatterplot | <code>sns.scatterplot(x="age", y="fare", data=df)</code>          |
|              | lineplot    | <code>sns.lineplot(x="time", y="sales", data=df)</code>           |
|              | relplot     | <code>sns.relplot(x="age", y="fare", hue="class", data=df)</code> |
| Categorical  | countplot   | <code>sns.countplot(x="class", data=df)</code>                    |
|              | barplot     | <code>sns.barplot(x="class", y="fare", data=df)</code>            |
|              | boxplot     | <code>sns.boxplot(x="class", y="fare", data=df)</code>            |

| Type           | Plot       | Code Example                                                      |
|----------------|------------|-------------------------------------------------------------------|
| Correlation    | violinplot | <code>sns.violinplot(x="class", y="fare", data=df)</code>         |
|                | heatmap    | <code>sns.heatmap(df.corr(), annot=True)</code>                   |
| Multi-variable | pairplot   | <code>sns.pairplot(df)</code>                                     |
|                | relplot    | <code>sns.relplot(x="age", y="fare", col="class", data=df)</code> |

**3. Basic Data Pre-processing Concepts** Data preprocessing is a crucial step in the data analysis and machine learning pipeline. It involves cleaning, transforming, and organizing raw data into a suitable format for analysis or modeling. Poor-quality data can lead to misleading insights, so proper preprocessing ensures reliability and performance.

### 3.1 Handling Missing Values (Removal, Imputation)

Missing data occurs when no value is stored for a variable in an observation.

Common Strategies

#### 1. Removal

- **Row-wise deletion:** Remove rows with missing values.
- **Column-wise deletion:** Remove columns with too many missing values.

**Pros:**

- Simple to implement
- No assumptions about missing data

**Cons:**

- Loss of information
- Can introduce bias if data is not missing at random

#### 2. Imputation

- **Mean/Median/Mode Imputation:** Replace missing values with statistical measures.
- **Forward/Backward Fill:** Use previous or next values (common in time series).
- **Interpolation:** Estimate missing values based on trends.
- **Model-based Imputation:** Use algorithms (e.g., regression, KNN).

**Example (Python):**

```
df['age'].fillna(df['age'].mean(), inplace=True)
```

### 3.2 Removing Duplicates

Duplicate records can distort analysis and model training.

## Methods

- Identify duplicates using all columns or a subset.
- Remove duplicates while keeping the first or last occurrence.

### Example:

```
df.drop_duplicates(inplace=True)
```

### Considerations:

- Ensure duplicates are truly redundant before removing.
  - Sometimes duplicates carry meaningful frequency information.
- 

## 3.3 Outlier Detection and Treatment

Outliers are extreme values that differ significantly from other observations.

### Detection Techniques

- **Statistical Methods:**
  - Z-score
  - IQR (Interquartile Range)
- **Visualization:**
  - Box plots
  - Scatter plots

### Treatment Methods

- Remove outliers
- Cap (winsorization)
- Transform data (log, square root)

### Example (IQR Method):

```
Q1 = df['value'].quantile(0.25)
Q3 = df['value'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['value'] >= Q1 - 1.5*IQR) & (df['value'] <= Q3 + 1.5*IQR)]
```

---

## 3.4 Encoding Categorical Variables

Machine learning models require numerical input, so categorical variables must be encoded.

### Techniques

- **Label Encoding:** Assign integer values to categories.
- **One-Hot Encoding:** Create binary columns for each category.
- **Ordinal Encoding:** Preserve order in categories.

### Example:

```
pd.get_dummies(df, columns=['color'])
```

**Considerations:**

- Avoid introducing false ordinal relationships.
  - One-hot encoding can increase dimensionality.
- 

### ***3.5 Feature Scaling (Normalization & Standardization)***

Scaling ensures that features contribute equally to model performance.

Methods

#### ***1. Normalization (Min-Max Scaling)***

- Scales values to a range (usually 0–1).

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
```

#### ***2. Standardization (Z-score Scaling)***

- Centers data around mean = 0 and standard deviation = 1.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

- Normalization: when data has no Gaussian distribution
  - Standardization: when data follows normal distribution
- 

### ***3.6 Datetime Processing***

Datetime data often requires transformation for analysis.

**Example:**

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
```

Applications:

- Time series analysis
  - Seasonal trend detection
- 

### ***\*\*\*3.7 Data Type Correction***

Incorrect data types can cause errors or incorrect analysis.

#### 0.0.4 Common Issues

- Numbers stored as strings
- Dates stored as objects
- Boolean values misrepresented

#### 0.0.5 Solutions

- Convert data types using appropriate functions

**Example:**

```
df['price'] = df['price'].astype(float)
```

**Benefits:**

- Improves memory efficiency
  - Ensures correct computations
- 

### 3.8 Feature Creation / Transformation

Creating new features can improve model performance.

Techniques

- **Mathematical Transformations:** log, square, square root
- **Interaction Features:** combine two or more variables
- **Binning:** group continuous values into categories
- **Text Features:** length, word count, TF-IDF

**Example:**

```
df['log_income'] = np.log(df['income'])
df['age_group'] = pd.cut(df['age'], bins=[0,18,35,60,100])
```

**Benefits:**

- Captures hidden patterns
  - Enhances predictive power
- 

[ ]: